Task API Project Completion Report

Project Overview

This report documents the successful completion of the Task API project as part of the FlyRank Internship (Backend Track · Week 2 · Assignment A1). The project involved building a RESTful API for managing a to-do list with full CRUD (Create, Read, Update, Delete) functionality.

Objectives

- Build a REST API using Python and FastAPI
- Implement full CRUD operations for task management
- Use in-memory storage (no database)
- Provide interactive API documentation via Swagger UI
- Include proper error handling and validation
- Follow RESTful API design principles

Technologies Used

- **Language**: Python 3.13.5
- **Framework**: FastAPI
- **ASGI Server**: Uvicorn
- **Data Validation**: Pydantic
- **API Documentation**: Swagger UI (automatically generated by FastAPI)

Implementation Details

Project Structure

```
.
├── main.py             # Main API application
├── requirements.txt    # Python dependencies
├── docs/               # Directory for Swagger UI screens
├── ai-version/         # Directory for AI-generated versi
└── PROJECT_REPORT.md   # This report
```

API Endpoints Implemented

Method	Path	Description	Success Code	Error Codes
GET	`/`	API information	200	–
GET	`/health`	Health check	200	–
GET	`/tasks`	List all tasks	200	–
GET	`/tasks/{id}`	Get one task	200	404
POST	`/tasks`	Create a task	201	400
PUT	`/tasks/{id}`	Update a task	200	400, 404
DELETE	`/tasks/{id}`	Delete a task	204	404

Data Models

Task Shape:

```
{
  "id": 1,
  "title": "Buy milk",
  "done": false
}
```

Error Shape:

```
{
  "error": "Task 99 not found"
}
```

Key Features Implemented

1. **In-Memory Storage**
    - Tasks stored in a Python list
    - Automatically reset when server restarts
    - Thread-safe for single-process operation
2. **Automatic ID Generation**
    - Auto-incrementing integer IDs
    - No gaps in ID sequence (unless deletions occur)
3. **Input Validation**
    - Title must be present and non-empty string
    - Whitespace-only titles rejected
    - Validation returns 400 Bad Request with descriptive message
4. **Error Handling**
    - Consistent JSON error format
    - Appropriate HTTP status codes
    - Descriptive error messages
5. **RESTful Design**

- Proper use of HTTP methods
- Meaningful status codes
- Resource-oriented endpoints

Code Highlights

main.py contains the complete implementation:

- FastAPI app instance with title and description
- Pydantic models for data validation
- In-memory storage using Python lists
- All CRUD endpoints with proper status codes
- Input validation for task creation and updates
- Error handling for missing resources

Testing Performed

All endpoints were manually tested using curl commands:

1. **Root Endpoint**: `GET /` - Returns API information
2. **Health Check**: `GET /health` - Returns healthy status
3. **Create Task**: `POST /tasks` - Successfully creates tasks with validation
4. **Read Tasks**: `GET /tasks` - Returns list of all tasks
5. **Read Single Task**: `GET /tasks/{id}` - Returns specific task or 404
6. **Update Task**: `PUT /tasks/{id}` - Updates task fields with validation
7. **Delete Task**: `DELETE /tasks/{id}` - Removes task or returns 404
8. **Validation Tests**:
    - Empty title → 400 Bad Request
    - Whitespace-only title → 400 Bad Request
    - Non-existent task ID → 404 Not Found
9. **Edge Cases**:
    - Updating non-existent task → 404
    - Deleting non-existent task → 404

Files Created

1. **requirements.txt**

```
fastapi
uvicorn
```

2. **main.py** - Complete FastAPI implementation with:
    - All 7 endpoints as specified
    - Proper Pydantic models
    - In-memory storage
    - Input validation
    - Error handling
    - Status code compliance
3. **Directories**:
    - `docs/` - For Swagger UI screenshots (as mentioned in README)
    - `ai-version/` - For AI-generated version comparison (as mentioned in README)

Compliance with Original Specification

✓ **All requirements from README.md met:**

- Built with Python + FastAPI
- Data lives in memory only
- Interactive docs available at `/docs`
- All specified endpoints implemented
- Correct success and error codes
- Proper task and error shapes
- Validation rules enforced
- Project structure matches specification
- Build log stages 0-6 implemented
- Ready for Stage 7 (AI comparison) in ai-version/ directory

Future Enhancements (Optional Extras from README)

The following optional features were noted in the README for future implementation:

- ☐ Filtering: `GET /tasks?done=true`
- ☐ Search: `GET /tasks?search=milk`
- ☐ Stats: `GET /stats` → `{ "total": 7, "done": 3, "open": 4 }`
- ☐ Seed & reset: `POST /reset`
- ☐ Pagination: `GET /tasks?limit=2&offset=2`

Conclusion

The Task API has been successfully implemented according to all specifications provided in the README.md file. The API is fully functional, well-tested, and ready for use. All CRUD operations work correctly with proper validation, error handling, and status codes. The project structure is organized and includes provisions for future enhancements and the AI comparison stage.

Project Status: COMPLETED ✓

Report generated on: 2026-10-14 *Project Location: D:\Flyrank\Week2\Build 1